Filling the grid

Six years ago while building a goofy side project I faced the following problem: how do I fit 48 squares on a screen so that they're as big as possible? I had just started high school and was programming to procrastinate on my math homework, hence I shamelessly copied a (partly wrong) solution off of StackOverflow and moved on.

There was only code – no correctness proof, no complexity analysis. Having just repeated and passed the first Basisprüfungsblock (finally), I learned with pain that this does not fly at ETH. Using my new hard earned knowledge I set out to make them myself, procrastinating this time on my math homework with... math.

Let's state the problem: place a $b \times a \in \mathbb{N}$ grid (rows x columns) into a $w \times h \in \mathbb{R}^+$ rectangle which fits $n \in \mathbb{N}$ squares such that the side length of the squares $s \in \mathbb{R}^+$ is maximized and the grid does not overflow the rectangle. Since the side length is maximized, the grid will fit in the rectangle's full width or height, or both if a perfect fit is possible. Formally:

$$
\begin{aligned}
S_w &:= \left\{ s \in \mathbb{R}^+ \;\middle|\; \exists a, b \in \mathbb{N} \,.\, n \leq ab \wedge as = w \wedge bs \leq h \right\} \\
S_h &:= \left\{ s \in \mathbb{R}^+ \;\middle|\; \exists a, b \in \mathbb{N} \,.\, n \leq ab \wedge as \leq h \wedge bs = h \right\} \\
s &:= \max(S_w \cup S_h).
\end{aligned}
$$

$s_w \in S_w$ is a *fit-width* solution, $s_h \in S_h$ a *fit-height* solution. The final solution s is the desired maximal side length. Our target is to build an algorithm to find s.

Observe that for any fixed b we can always take $a = \left\lceil \frac{n}{b} \right\rceil$ to obtain a grid which fits n squares, since clearly $n \leq ab$. For this grid (a, b) let $s' = \min \left\{ \frac{w}{a}, \frac{h}{b} \right\}$ its corresponding side length; it's easy to see that $s' \in S_w \cup S_h$. We can just test all $1 \leq b \leq n$ and pick the maximum s' to obtain the solution s:

```
function fillGrid(n, w, h) {
    let s = 0;
    for (let b = 1; b <= n; b++) {
        s = Math.max(s, Math.min(w / Math.ceil(n / b), h / b))
    }
    return s
}
```

Used Javascript so someone's offended, golfed the code so the article fits, asymptotic complexity is $\mathcal{O}(n)$. To improve this, let $k = \left\lceil \frac{n}{a} \right\rceil$. If $k \leq \sqrt{n}$ there are obviously $\mathcal{O}(\sqrt{n})$ distinct values of k. If $k > \sqrt{n}$:

$$
\left\lceil \frac{n}{a} \right\rceil > \sqrt{n} \implies \frac{n}{a} + 1 > \sqrt{n} \implies a < \sqrt{n} + \frac{1}{\sqrt{n}} \leq \mathcal{O}(\sqrt{n}).
$$

Therefore there are $\mathcal{O}(\sqrt{n})$ distinct values of $\left\lceil \frac{n}{a} \right\rceil$, meaning with clever incrementing of b we could achieve $\mathcal{O}(\sqrt{n})$ complexity. But for my grid with $n = 10^{20}$ squares this isn't good enough, so...

Geht es besser?

The solution statement as it stands juggles a lot of variables. We can rewrite the grid dimension constraints using only a, b, w, h:

$$\begin{aligned} as_w = w \land bs_w \leq h &\iff a \geq \frac{w}{h} b \land s_w = \frac{w}{a} \\ bs_h = h \land as_h \leq w &\iff b \geq \frac{h}{w} a \land s_h = \frac{h}{b}. \end{aligned}$$

s is now dependent on the grid and rectangle dimensions. Using $r := \frac{w}{h}$, the rectangle's aspect ratio:

$$\begin{aligned} a_w &:= \min \left\{ \, a \in \mathbb{N} \mid \exists b \in \mathbb{N} \,.\, n \leq ab \land a \geq rb \, \right\} \\ b_h &:= \min \left\{ \, b \in \mathbb{N} \,\middle|\, \exists a \in \mathbb{N} \,.\, n \leq ab \land b \geq \frac{a}{r} \, \right\} \\ s &= \max \left\{ \, \frac{w}{a_w}, \frac{h}{b_h} \, \right\}. \end{aligned}$$

a_w is the column count of the *fit-width* grid, b_h the row count of the *fit-height* grid. They are minimized since the side length is inversely proportional. The side length is a trivial result of the grid dimensions; only the algorithm for finding a_w and b_h remains to be developed.

Let's take a look at a fit-width grid with dimensions $(a_w, b_w) \in \mathbb{N}^2$. b_w can be abstracted away:

$$\begin{aligned} n \leq a_w b_w \land a_w \geq r b_w &\iff \frac{n}{a_w} \leq b_w \leq \frac{a_w}{r} \\ &\iff \left\lceil \frac{n}{a_w} \right\rceil \leq \frac{a_w}{r} \\ &\iff a_w \geq r \left\lceil \frac{n}{a_w} \right\rceil. \end{aligned}$$

With an analogous derivation for b_h, we get:

$$a_w = \min \left\{ \, a \in \mathbb{N} \,\middle|\, a \geq r \left\lceil \frac{n}{a} \right\rceil \, \right\}, \quad b_h = \min \left\{ \, b \in \mathbb{N} \,\middle|\, rb \geq \left\lceil \frac{n}{b} \right\rceil \, \right\}.$$

Since the second comparison term is non-increasing in a (respectively in b), it follows that every $a > a_w$ (and $b > b_h$) would satisfy the solution condition. The algorithm idea is thus the following: starting from an initial guess $a_0 \leq a_w$ (and $b_0 \leq b_h$) test every value until the solution condition holds, i.e. loop while:

$$a < r \left\lceil \frac{n}{a} \right\rceil, \quad rb < \left\lceil \frac{n}{b} \right\rceil.$$

Only the initial guesses a_0 and b_0 remain to be made. From the solution condition of a_w:

$$a_w \geq r \left\lceil \frac{n}{a_w} \right\rceil \implies {a_w}^2 \geq rn \implies a_w \geq \sqrt{rn}.$$

Since $a_w \in \mathbb{N}$ it can therefore be no smaller than $a_0 = \lceil \sqrt{rn} \rceil$. Likewise, $b_w \geq b_0 = \left\lceil \sqrt{\frac{n}{r}} \right\rceil$. Using the loop condition above, an algorithm can finally be built:

```
function fillGrid(n, w, h) {
    const r = w / h

    let a = Math.ceil(Math.sqrt(r * n))
    for (; a < r * Math.ceil(n / a); a++);

    let b = Math.ceil(Math.sqrt(n / r))
    for (; r * b < Math.ceil(n / b); b++);

    return Math.max(w / a, h / b)
}
```

To analyze the runtime complexity, observe that since $a_0 \leq a < r \left\lceil \frac{n}{a} \right\rceil$, a will be incremented at most $\left\lceil r \left\lceil \frac{n}{a} \right\rceil - a_0 \right\rceil$ times. Moreover, since a increases, $\left\lceil \frac{n}{a} \right\rceil \leq \left\lceil \frac{n}{a_0} \right\rceil$. Hence an upper bound on the iteration count is:

$$\begin{aligned}
\left\lceil r \left\lceil \frac{n}{a} \right\rceil - a_0 \right\rceil &< r \left\lceil \frac{n}{a_0} \right\rceil - a_0 + 1 \\
&< r \frac{n}{a_0} + r - a_0 + 1 \\
&= r \frac{n}{\lceil \sqrt{rn} \rceil} + r - \lceil \sqrt{rn} \rceil + 1 \\
&< \frac{rn}{\sqrt{rn}} + r - \sqrt{rn} + 1 \\
&= r + 1.
\end{aligned}$$

This means at most $\lfloor r \rfloor + 1$ iterations. This bound is tight: for $n = 33, r = \frac{1}{8.3}$ the fit-width loop does $1 = \lfloor r \rfloor + 1$ iteration, reaching the upper bound. Similarly for the second loop we get a tight upper bound of $\left\lfloor \frac{1}{r} \right\rfloor + 1$ iterations. The runtime is therefore $\mathcal{O}(R)$ with $R = \max\left\{ r, \frac{1}{r} \right\}$. One could binary search over $\left[a_0, a_0 + \lfloor r \rfloor + 1\right]$ and $\left[b_0, b_0 + \left\lfloor \frac{1}{r} \right\rfloor + 1\right]$ to reduce complexity to $\mathcal{O}(\log R)$.

My $n = 10^{20}$ grid is a piece of cake now, but my $r = 2 \uparrow\uparrow 6$ grid still requires around 10^{19700} iterations. There are 10^{80} atoms in the observable universe. *Geht es besser?*

This problem is a particular case of *integer programming*: find minimal/maximal integer solution to a problem satisfying constraints. It is an NP-complete problem, so there are two ways to solve it: heuristics or exact algorithms. Up until now we've tried two heuristic methods, one giving $\mathcal{O}(\sqrt{n})$, another $\mathcal{O}(\log n)$.

Our problem is very simple – just one variable (either the fit-width or the fit-height solution) and two constraints – meaning the complexity of the exact algorithms remains low. Sadly they don't help: any such algorithm (e.g. Lenstra's) give us $\mathcal{O}((\log R)^{\mathcal{O}(1)})$, which isn't asymptotically better than what we already have. Could we find a better heuristic?

Let's return to the initial algorithm: iterate through all possible b. Can we further restrict the range of b? For a fit-height solution s_h we know that the minimum b_h is greater than $b_0 = \left\lceil \sqrt{\frac{n}{r}} \right\rceil$. Suppose now that $r \geq 1$ and that $b_h = b_0 + 1$ is *not* a fit-height solution. This means the loop condition from above is true, which implies:

$$\begin{aligned} rb_h < \left\lceil \frac{n}{b_h} \right\rceil &\implies rb_0 + 1 \leq r(b_0 + 1) < \left\lceil \frac{n}{b_0 + 1} \right\rceil \\ &\implies rb_0 + 1 < \frac{n}{b_0 + 1} + 1 \\ &\implies \sqrt{{b_0}^2 + b_0} < \sqrt{\frac{n}{r}} \leq b_0. \end{aligned}$$

A contradiction. It follows that for $r \geq 1$ the minimal $b_h \in \{\, b_0, b_0 + 1 \,\}$. Let now $b_w = b_h - 1$ and $a_w = \left\lceil \frac{n}{b_w} \right\rceil$. Since $n \leq a_w b_w$ but $b_w < b_h$ by minimality of b_h it must be that $a_w > rb_w$, meaning (a_w, b_w) is a fit-width solution. For any other fit-width solution (a, b) with $a < a_w$ and side length $s = \frac{w}{a}$:

$$\begin{aligned} a < \left\lceil \frac{n}{b_h - 1} \right\rceil \leq \left\lceil \frac{ab}{b_h - 1} \right\rceil &\implies a < \frac{ab}{b_h - 1} \implies b \geq b_h \\ &\implies a \geq rb \geq rb_h \implies \frac{w}{a} \leq \frac{h}{b_h} \\ &\implies s \leq s_h. \end{aligned}$$

No such fit-width (a, b) is better than the fit-height (a_h, b_h). Hence the only useful fit-width solution is $b_w = b_h - 1 \in \{\, b_0 - 1, b_0 \,\}$. In conclusion for $r \geq 1$ the optimal solution s will *always* correspond to a grid with $b \in \{\, b_0 - 1, b_0, b_0 + 1 \,\}$ rows.

Lastly, notice the symmetry of the problem with respect to r. If one takes $r' := \frac{1}{r}$ intuitively this just rotates the original rectangle 90°. Anything proven for $r \geq 1$ applies to $r < 1$ with flipped dimensions. Using the result above, for $r < 1$ the optimal grid (a, b) must have $a \in \{\, a_0 - 1, a_0, a_0 + 1 \,\}$ and $(a, b) = (b', a')$, where (a', b') is the optimal grid for r'.

With this we have exhaustively covered the input domain. The $\mathcal{O}(1)$ algorithm we've all been waiting for is...

```
function fillGrid(n, w, h) {
    const r = w / h
    if (r < 1) {
        return fillGrid(n, h, w)
    }
    const s = (b) => Math.min(w / Math.ceil(n / b), h / b)
    const b0 = Math.ceil(Math.sqrt(n / r))
    return Math.max(s(b0 - 1), s(b0), s(b0 + 1))
}
```

Besser geht es nicht.

If you've found the proofs hard to follow, a geometric representation helps: plot the functions $\frac{n}{x}$ and $\frac{x}{r}$ and the solution points (a, b) and use the graph for interpretation. Here's a Desmos: desmos.com/calculator/z0ubu4uih8.